AI在大型复杂软件工程中的落地瓶颈与确定性底座建设讨论 - 摘要

16:05 2026/10/23
上海市, CN
由AI生成，请在使用前核实

概览

本次讨论围绕AI与Code Agent在大型复杂软件工程中的落地瓶颈展开，分析了多端架构代码腐化、过时文档规则及客户端验证难以扩展对AI研发交付的制约。发言人指出交付速度取决于“可信反馈底座的AI能力次方”，并阐述了B站通过Mono Repo与统一构建验证等确定性工程底座来保障交付确定性的思路。

要点

- 大型软件工程中AI不是短板的补齐，而是短板的放大器，会将工程原有的优缺点放大100倍。
- 交付速度的核心公式为“可信反馈底座的AI能力次方等于能放心的交付速度”，其中反馈底座与AI本身无关，属于工程确定性基础设施。
- B站总App编译包含14.3万个action，串行依赖深度达64层，变更影响无法单靠人力感知。
- 客户端受限于原生编译和模拟器等待机制，验证环节无法像后端拆分API那样线性扩展，导致代码写得快但Review和验证滞后，产生大量无人敢合入的代码积压。
- 针对多端各自腐化的问题，B站将移动端统一集成到Bazel构建体系，依托Mono Repo提供完整的上下文与原子能力，并按“约束、定位、验证、度量”四个顺序构建工程底座。

详情

- AI在大型复杂软件工程中的放大效应与挑战
  - 严肃工程中的短板放大: AI对个人开发者或Demo研究是能力补齐，但在大型严

肃工程中会将原有的优点和缺点同时放大100倍。 00:04:14

- 规则破坏性放大: 规则多写在文档中靠人工记忆，经AI放大后规则的破坏性同步剧增，在传统分端协作下尤为严重。 00:04:49
- 协同与流水线压力: 500人规模的工程面临文档对齐、CI流水线吞吐能力以及代码Review积压的现实挑战，Code Agent实质上放大了500人并发协作的工程复杂度。 00:00:19
- B站App编译规模与串行链路:
  - 仓库中九成以上代码存在依赖关联，单行代码变更的实际影响面无法靠人力感知。 00:02:24
  - 总App编译包含14.3万个action，涵盖Gradle任务与C/CPP编译等。 00:02:24
  - 关键编译路径深度达64层且必须串行执行，阻碍并发扩展。 00:02:24
- 组织结构全栈化重构: B站推行全栈架构重构，上层涵盖国内与出海业务交付，中层处理大流量、高并发与高一致性后端，底层提供基础架构支持。 00:02:34

- 多端架构现状与代码腐化对AI的影响
  - 架构与构建工具差异导致的代码腐化:
    - Android端受限于Gradle构建机制，直播间模块演变为包含2251个文件、30万行的巨大单体模块。 00:11:03
    - iOS端因2018年改造迁移至Bazel，直播间项目被拆分为957个子模块。 00:11:16
    - Web端作为NPM模块代码量达40万行，且异步机制中仍有约五分之一代码保留Promise then模式。 00:10:53
  - AI对现有代码库的顺从性: Code Agent进入仓库后会顺应既有结构与代码习惯进行学习和生成，不会主动重构或限制超大文件的膨胀，甚至会在人工尝试改造时提出质疑。 00:11:31

    - 多端协同中的长尾错误: 传统多端开发中常出现iOS端JSON key大小写错误、Android端序列化库使用不一致以及Web端漏对齐需求等问题，高度依赖PMO与Jira跟进而耗费大量时间。 00:23:03

- 过时文档与规则对AI生成的负面影响
    - 错误注释对AI的误导: iOS画中画（PIP）模块中一行关于生命周期相差300毫秒的过时注释导致AI持续产生幻觉，排查耗时达一周。 00:12:22
    - 规则文件（Harness）失效与维护负担:
        - GPT模型升级后官方适配文档建议清空并重写所有Code MD与Agent MD规则文件，避免老规则拖后腿。 00:13:40
        - 开发者为多端一致性或敏感信息红线手工追加的Agent规则补丁，在运行三至五个月后会快速腐化并导致AI推理偏差。 00:15:16
- 客户端编译与验证无法扩展的瓶颈
    - 原生编译与运行时的固有延迟: 基于LLVM的Native代码编译工作量远大于解释型语言或JVM编译；Android尚可基于Linux虚拟化内核，但iOS必须等待Xcode模拟器，导致客户端验证无法像后端多实例并发那样线性扩展。 00:16:10
    - 生产与交付鸿沟: 编译、运行与测试环路缺乏百倍加速能力，导致AI快速产出的代码因缺乏可信验证而形成无人敢合入的积压。 00:18:53
- 确定性底座与Mono Repo的价值
    - 交付公式与底座定位: 能放心的交付速度由“可信反馈底座的AI能力的次方”决定，底座本身是纯工程确定性建设，与AI能力本身无关。 00:19:54
    - Mono Repo的价值跃迁: 在AI代码生成提速的背景下，Mono Repo从过去仅用于提升编译速度转变为支撑AI获取完整上下文的关键设施。 00:20:27
- B站工程底座建设与交付确定性解决思路
    - 统一工程环境与上下文供给: B站采用Mono Repo架构，将移动端全部集成至Bazel，将代码关系与环境验证整合到同一个代码引擎上下文中，仅向AI提供

原子能力而不使用分立的Survey Agent。 00:20:50

- 交付确定性闭环流程: 围绕“怎么约束、怎么定位、怎么验证、怎么度量”四个环节按严格顺序展开建设，确保代码负责人敢于合并代码与发布K8S上线。 00:21:17